# *Singly Linked List – Insert At A Position And Time Complexity.*

## *Program*:

```
#include <iostream>
#include <stack>
using namespace std;

typedef struct ListNode
{
    int data;
    struct ListNode *next;
} Node;

Node *head;

void createEmptyList(Node **head)
{
    *head = nullptr;
    cout << "Empty list created!!!" << endl;
}

void insertAtBeginning(Node **head, int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;
    if (*head == nullptr)
    {
        newNode->next = nullptr;
        *head = newNode;
    }
```

```
    else
    {
        newNode->next = *head;
        *head = newNode;
    }
}
void insertAtEnd(Node **head, int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;
    newNode->next = nullptr;
    if (*head == nullptr)
    {
        *head = newNode;
    }
    else
    {
        Node *current = *head;
        while (current->next != nullptr)
        {
            current = current->next;
        }
        current->next = newNode;
    }
}

void insertAtPosition(Node **head, int data, int position)
{
    Node *newNode = (Node *)malloc(sizeof(Node));

    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
```

```
        return;
    }

    newNode->data = data;

    if (position == 1)
    {
        newNode->next = *head;
        *head = newNode;
        return;
    }

    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }

    if (position < 1)
    {
        cout << "Error: Invalid position." << endl;
        return;
    }
    else
    {
        Node *current = *head;
        int i = 1;
        while (i < position - 1 && current != nullptr)
        {
            current = current->next;
            i++;
        }
        if (current == nullptr)
        {
            cout << "Error: Invalid position." << endl;
            return;
        }
```

```
        else
        {
            newNode->next = current->next;
            current->next = newNode;
        }
    }
}


int main()
{
   createEmptyList(&head);
    while (1)
    {

        cout << "1. Insert at the beginning" << endl;
        cout << "2. Insert at the end" << endl;
        cout << "3. Insert at a position,indexing node from 1 to N" << endl;

        int choice;
        cout << "Enter your choice: ";
        cin >> choice;
        switch (choice)
        {
        case 1:
        {
            int data;
            cout << "Enter the data: ";
            cin >> data;
            insertAtBeginning(&head, data);
            break;
        }
      case 2:
      {
            int data;
            cout << "Enter the data: ";
            cin >> data;
            insertAtEnd(&head, data);
```

```
            break;
      }
      case 3:
        {
            int data,position;
            cout << "Enter the data: ";
            cin >> data;
            cout << "Enter the position: ";
            cin >> position;
            insertAtPosition(&head, data, position);
            break;
          }

       default:
        {
            cout << "Invalid choice." << endl;
            break;
        }
        }
    }
  return 0;
}
```

## ***Program Analysis***:

```
    Node *newNode = (Node *)malloc(sizeof(Node));
```

→ *The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.*

→ *The size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

→ *The size of the Node structure itself is determined by the sum of the sizes of its members*:

→ *data(an integer) is typically 4 bytes.*

→ *next(a pointer to another Node) is typically 8 bytes.*

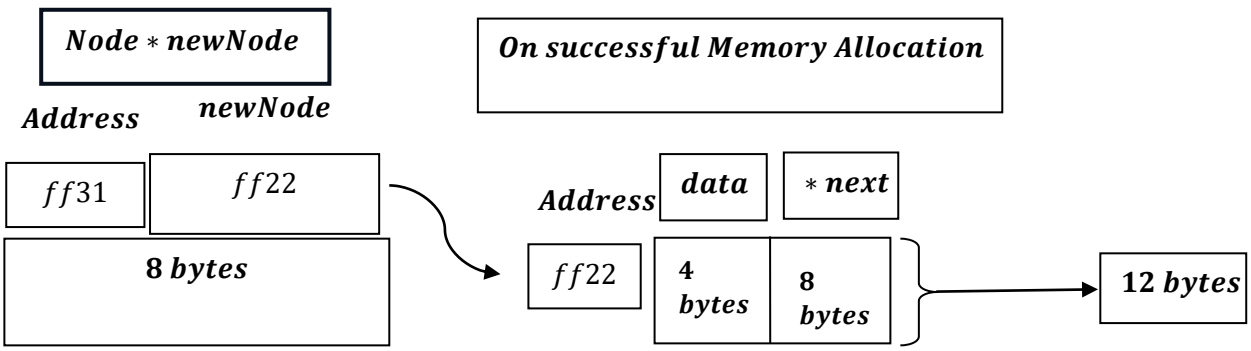
→ *Therefore, the size of the Node structure is typically 12 bytes( 4 bytes for data + 8 bytes for next).*

```
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
```

*What does it mean?*

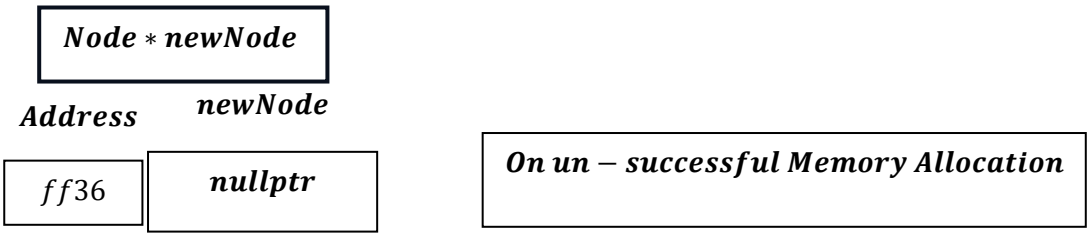
→ *newNode will try to allocate memory for data and next pointer variable i.e. typically 8 bytes, though for variable data and structure Node's pointer variable next in total 12 bytes, but as discussed earlier, the size of a pointer (newNode) is typically 8 bytes on most modern 64 − bit systems, regardless of the size of the structure it points to.*

*Say,*




*But on Un − Successful memory allocation:*

→ *if the allocation fails due to insufficient memory or other reasons, malloc will return a null pointer.*



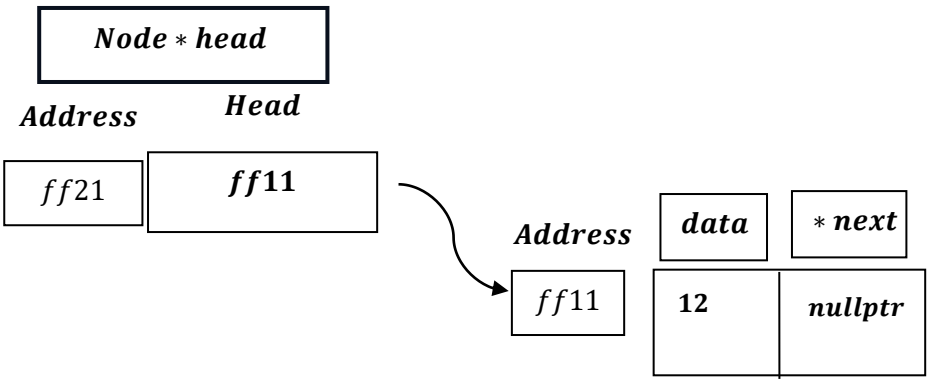

*Now, if the value of the pointer variable newNode is nullptr, then:*

*One may output* : `Overflow *or Memory Allocation Failed as a message.*`

*Now, we move from Node 1 to Node N .*

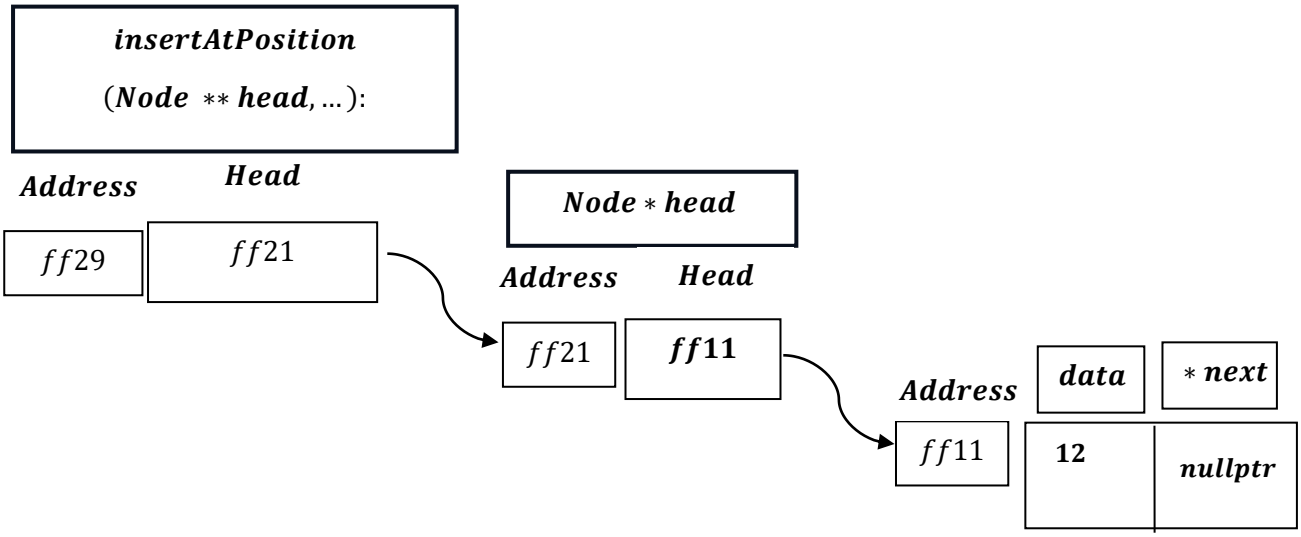
```
    case 3:
        {
            int data,position;
            cout << "Enter the data: ";
            cin >> data;
            cout << "Enter the position: ";
            cin >> position;
            insertAtPosition(&head, data, position);
            break;
        }
```

*Say, data is 20 , and position is 1 hence we pass through function call:*



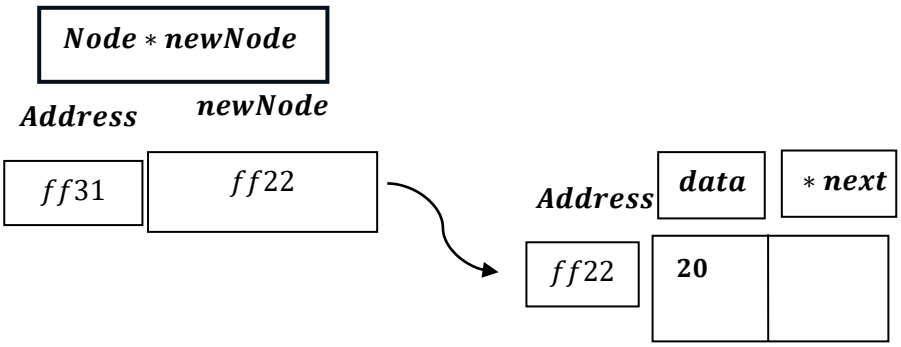
$insertAtPosition(ff21, 20, 1)$ ;

```
void insertAtPosition(Node **head, int data, int position) {…}
```




```
    newNode->data = data;
```

*The newNode pointer of Node structure points to address of data , assigns the value of data i.e. 20 here.*



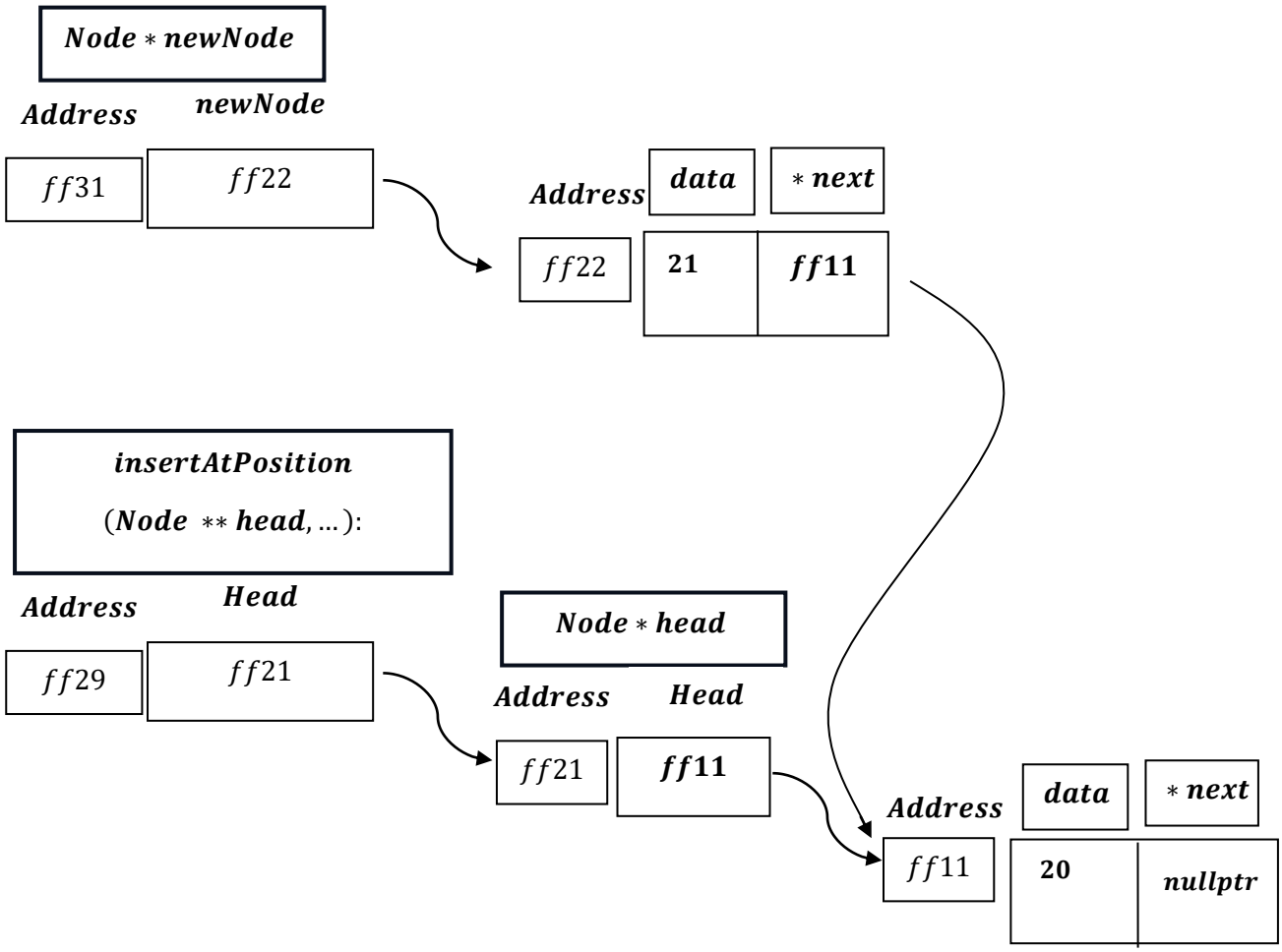

```
   if (position == 1)
       {
        newNode->next = *head;
```

```
        *head = newNode;
        return;
    }
```

$For\ newNode \rightarrow next = *head; i.e. next\ pointer\ assigned\ to\ the\ value\ of$
$head\ pointer\ of\ Node\ structure. Here\ value\ is: ff11.$

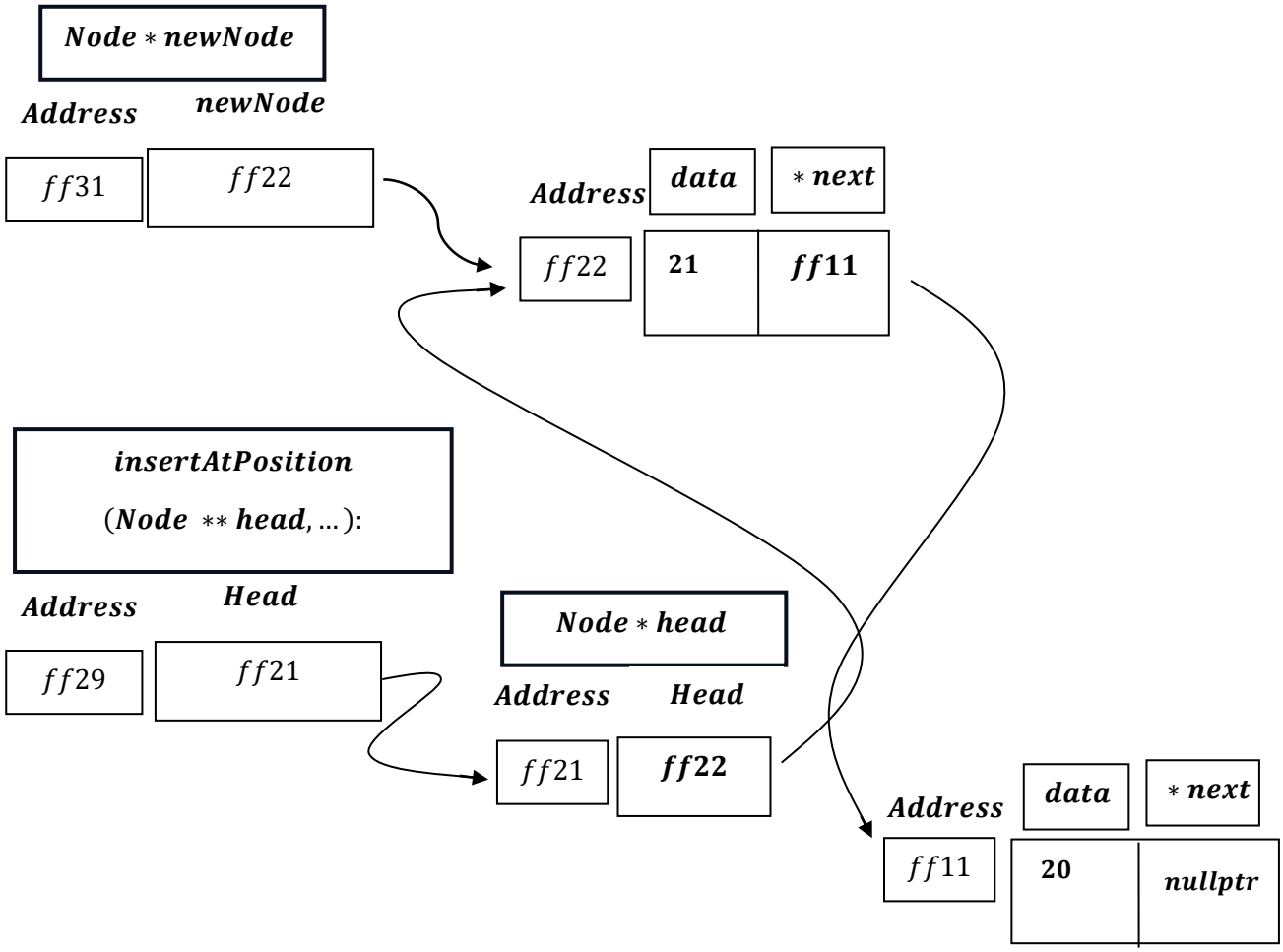
$Why\ ff11?\ \ newNode \rightarrow next = *head\ , here\ head\ used\ is\ of\ local\ pointer\ to\ pointer$
$i.e. Node\ **\ head, and\ as\ *\ head\ is\ used, it\ will\ first\ derefer\ from\ Node\ **\ head$
$and\ refer\ us\ to\ Node\ *\ head, which\ will\ cause\ next\ pointer\ to\ store\ value\ contained$
$in\ Node\ *\ head\ \ i.e. ff11\ , address\ of\ Head\ node\ to\ which\ Node\ *\ head\ pointer$
$points\ to.$

```
        newNode->next = *head;
```

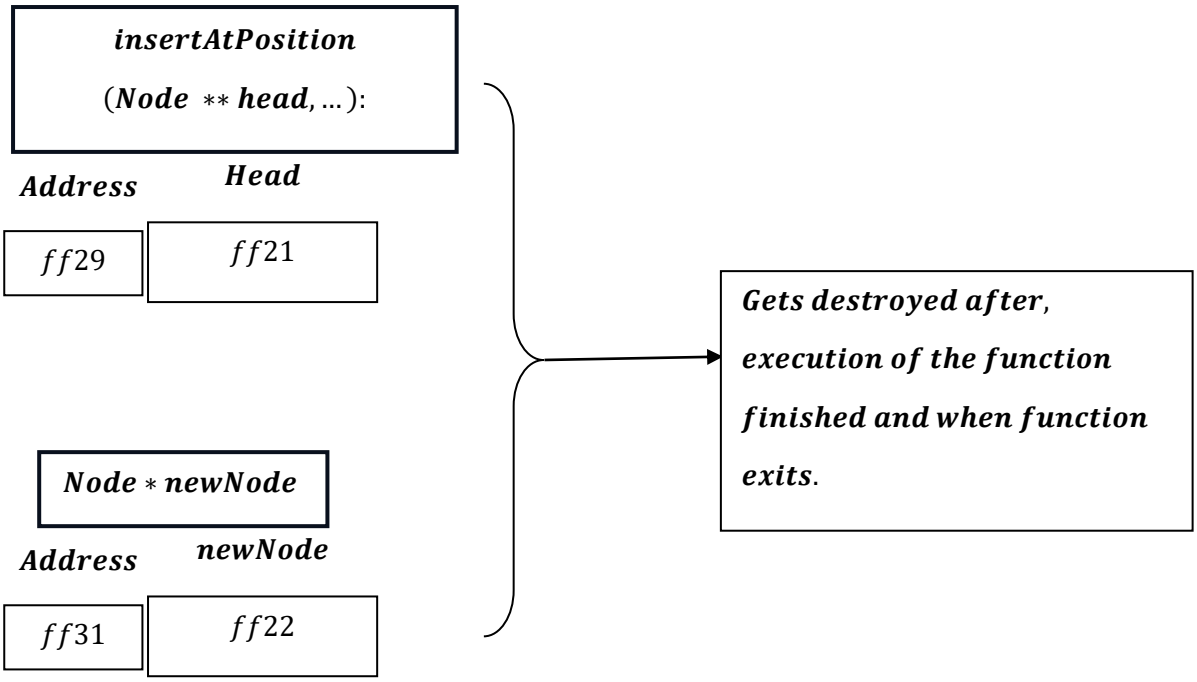


```
        *head = newNode;
```

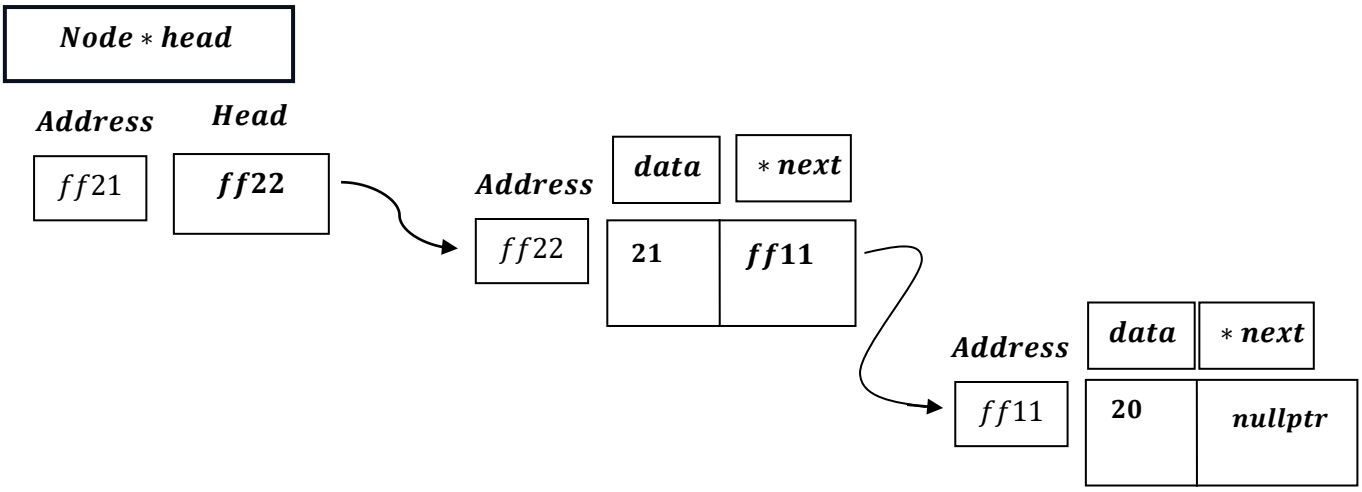
*What does it mean? It means , here head variable used is of Node ∗∗ head , temporary pointer to pointer variable in function, As we use ∗ head , it dereference from Node ∗∗ head and refer us to Node ∗ head, global pointer to which Node ∗∗ head points to. And the line of code ` ∗ head = newNode` change the value of global pointer Node ∗ head , due to dereferencing of Node ∗∗ head to Node ∗ head and Node ∗ head gets assigned to address of newNode pointer variable , which stored as a value in Node ∗ head[here , ff11]. And the value i.e. the address of Node ∗ head[here, ff21] stored in Node ∗∗ head will remain unchanged and unaltered.*




*Now, as Node ∗ newNode and Node ∗∗ head are temporary variable, therefore gets destroyed after function execution get finished .*



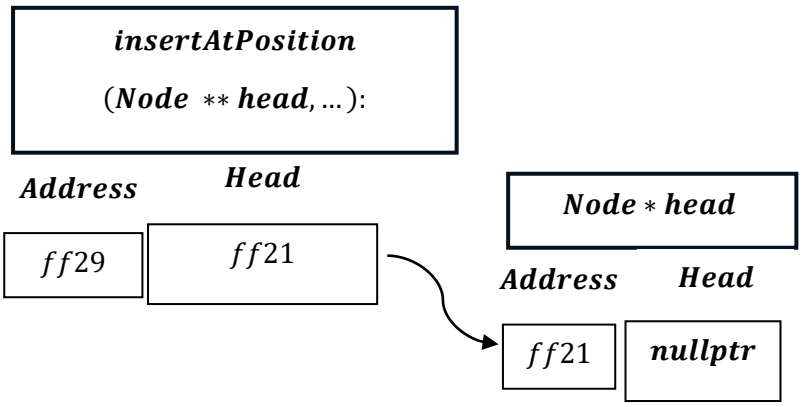

***And we are left with***:




---

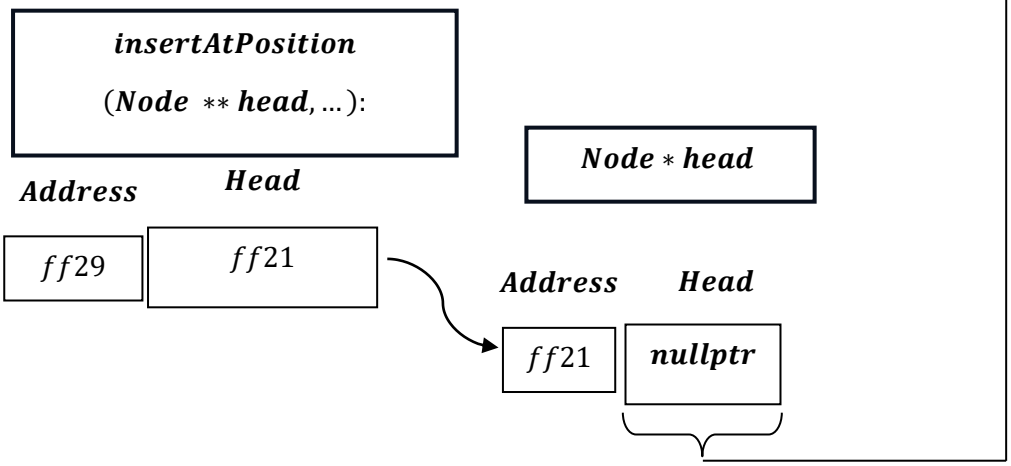
*Now, we came up with some error handling scenarios*:

```
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
```




$if(*head == nullptr)$ *if it is true*

*If true then Why true ?*
*Ans: Though * head which is of Node ** head*
*temporary pointer to pointer variable of the function*
*, as * head is used , it dereference from Node ** head and*
*refer to Node * head global pointer to which Node ** head*
*is pointing to and then its value is checked is*
*whether equal to nullptr or not ? This occurs during execution*
*and compilation of the insertAtPosition function.*



*Then , it prints a message : `Error: List is empty.`*

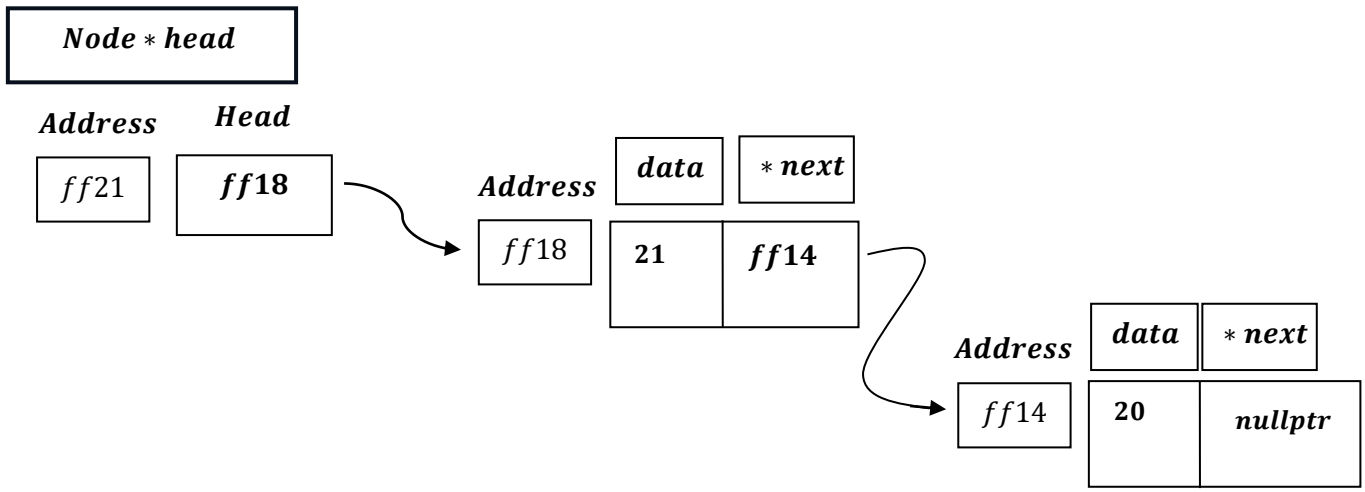
```
    if (position < 1)
    {
        cout << "Error: Invalid position." << endl;
        return;
    }
```

*If position is less than 1 , then it returns message : ` Error : Invalid position.`*

---

*Now , if position is greater than 1 and head doesnot contain nullptr i.e.*

*we have:*




```
    else
    {
        Node *current = *head;
        int i = 1;
        while (i < position - 1 && current != nullptr)
        {
            current = current->next;
            i++;
```

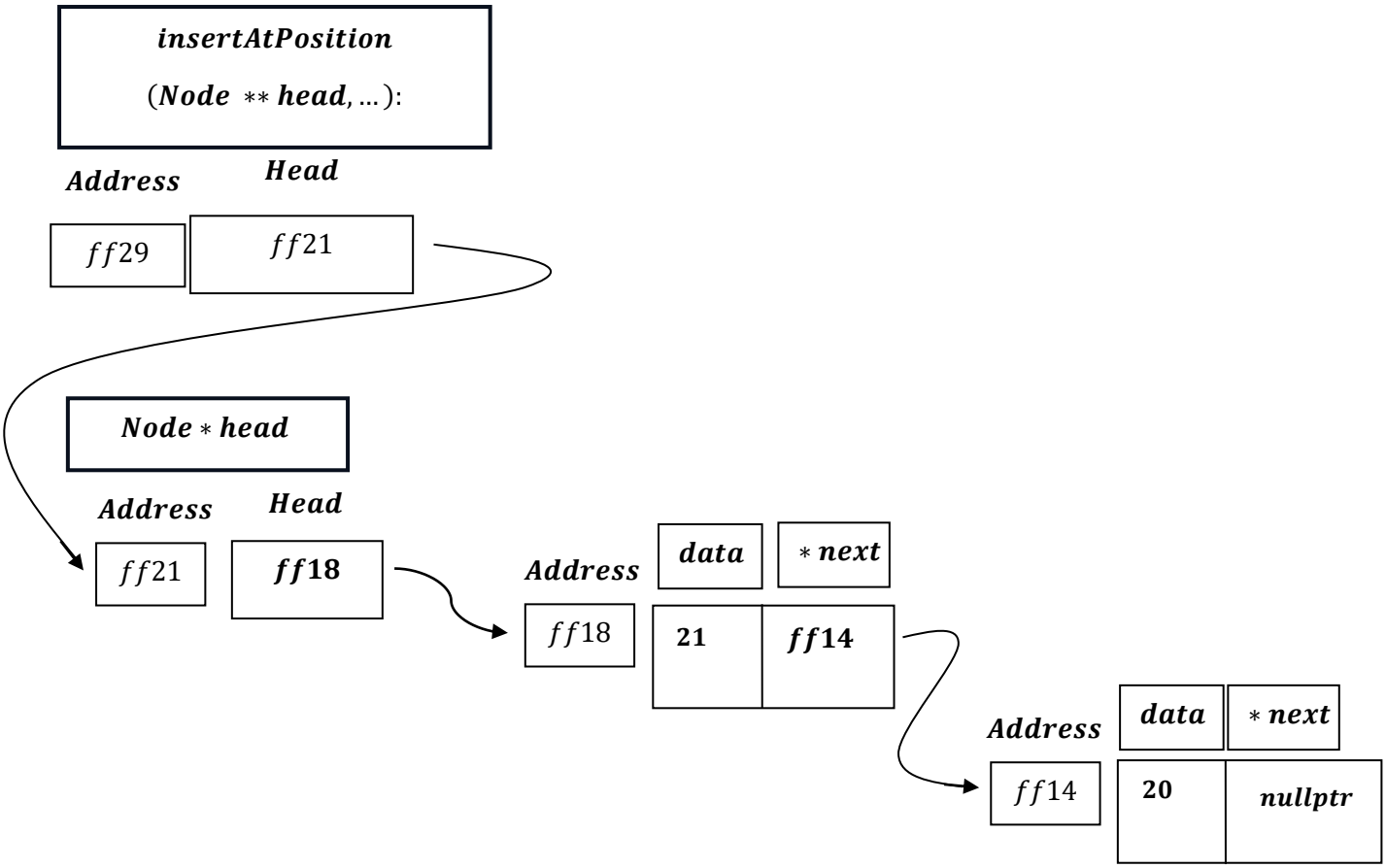
```
        }
        if (current == nullptr)
        {
            cout << "Error: Invalid position." << endl;
            return;
        }

        else
        {
            newNode->next = current->next;
            current->next = newNode;
        }
    }
```

$Now, say\ we\ pass\ data : 19\ at\ position : 3.$

```
    case 3:
        {
            int data,position;
            cout << "Enter the data: ";
            cin >> data;
            cout << "Enter the position: ";
            cin >> position;
            insertAtPosition(&head, data, position);
            break;
        }
```

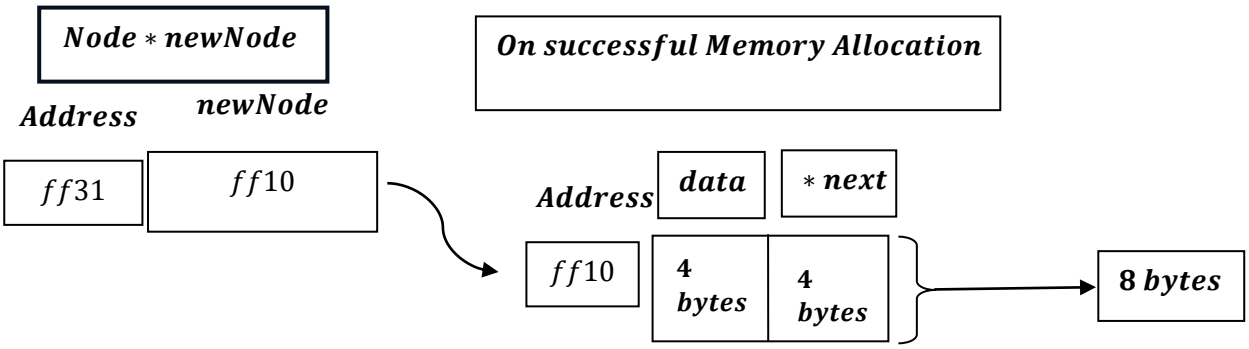
$InsertAtPosition(ff21, 19, 3)\ ;$

```
void insertAtPosition(Node **head, int data, int position) {…}
```



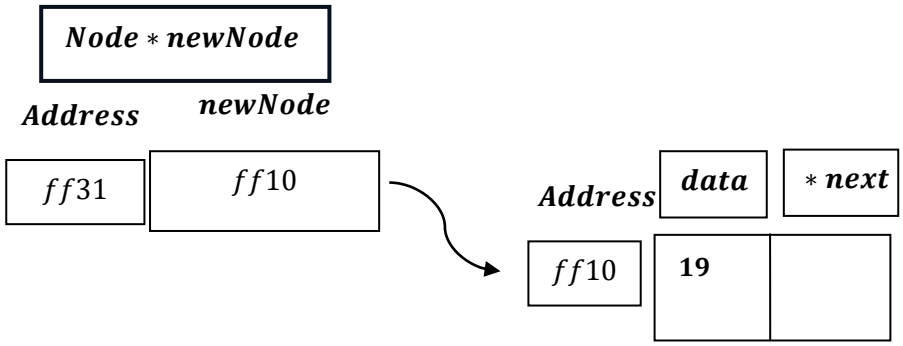

***Next***:

```
    Node *newNode = (Node *)malloc(sizeof(Node));
```




***Going with successful memory allocation.***

***Next***:

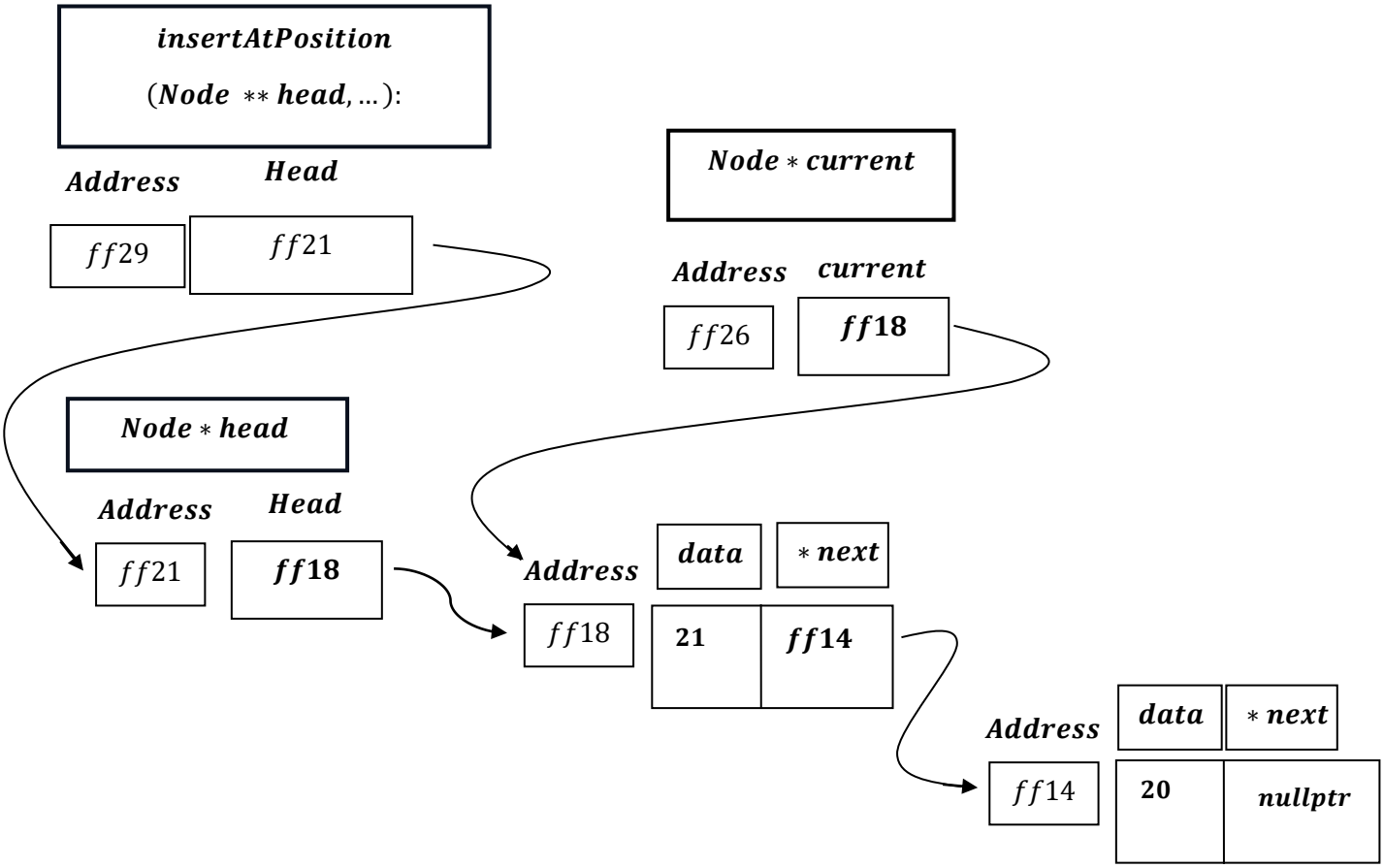



***As***: $\boldsymbol{InsertAtPosition(ff21, 19, 3)}$ ;

***Now, coming to else part***:

```
        Node *current = *head;
```

***current is a pointer of Node structure , whose value is assigned with pointer head's value.***



$Why\ ff18\ ?\ ,when\ we\ are\ using * head, i.e. here\ head\ used\ is\ of\ local\ pointer$
$to\ pointer\ i.e. Node ** head\ , and\ as * head\ is\ used, it\ will\ first\ derefer\ from$
$Node ** head\ and\ refer\ us\ to\ Node * head\ , which\ will\ cause\ current\ pointer\ to$
$store\ value\ contained\ in\ Node * head\ \ i.e. ff18\ , address\ of\ Head\ node\ to\ which$
$Node * head\ pointer\ points\ to.$

```
int i = 1;
while (i < position - 1 && current != nullptr)
{
    current = current->next;
    i++;
}
```

$Our\ position\ is : 3\ , hence\ position - 1\ will\ become\ 3 - 1 = 2\ and$
$i < position - 1\ i.e. i < 2\ , hence\ i\ will\ looped\ for\ once(1)\ only\ and\ at\ this$
$rate\ , current\ will\ never\ become\ nullptr\ as\ current \rightarrow next\ will\ not\ reach$
$the\ end\ node\ which\ contain\ next\ pointer\ value :\ \ nullptr\ , if$
$position\ is\ \ between\ 1\ to\ N + 1\ i.e. suppose\ length\ of\ linked\ list\ is\ 8\ , then$
$node\ can\ be\ inserted\ at\ position : 1, 2, \ldots., 9\ \ and\ current\ will\ never$
$carry\ `nullptr`\ .$

$But\ if\ position\ is\ 10\ it\ will\ return\ Nullptr, loop\ will\ get\ terminated\ .$

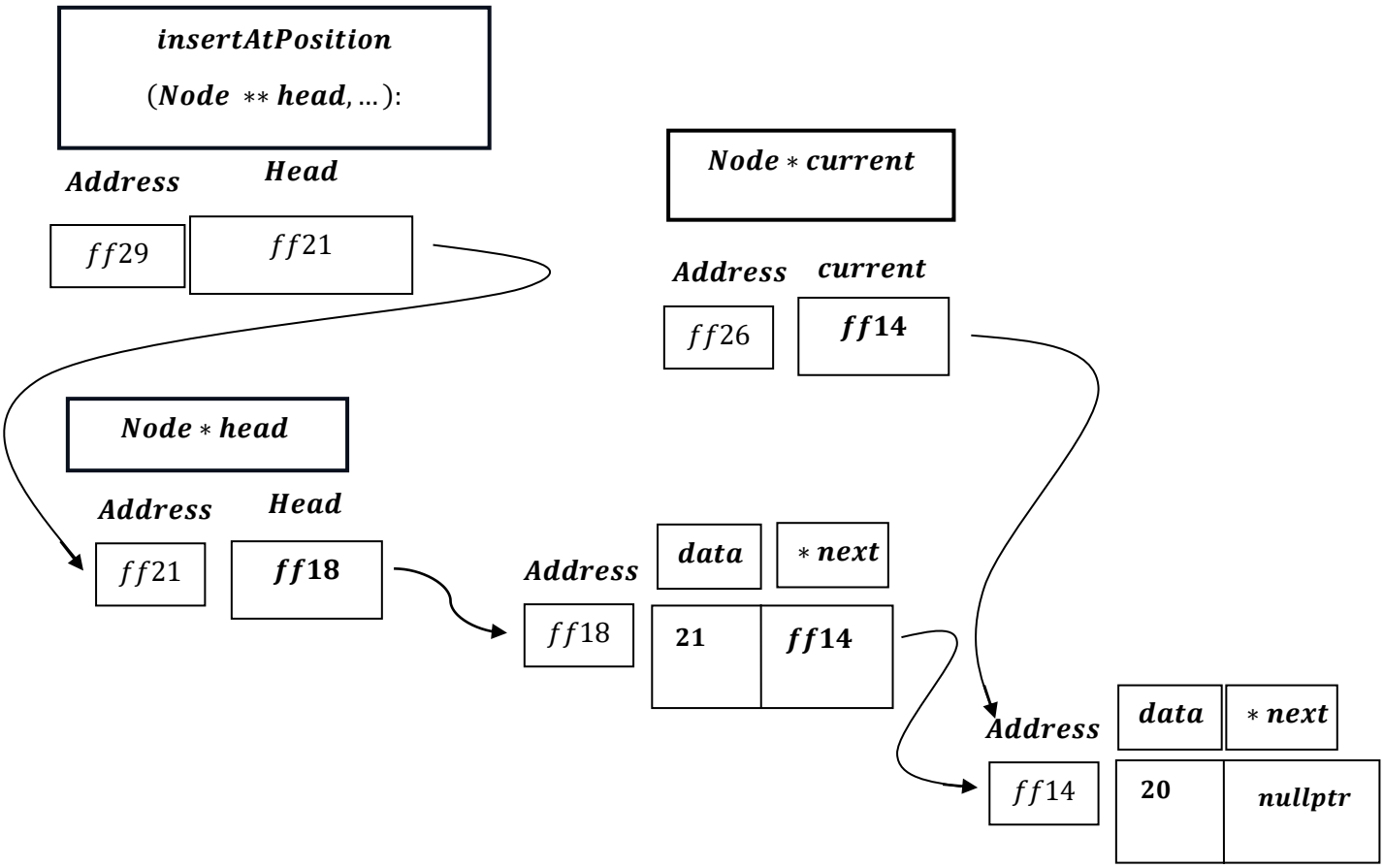
$Now\ our\ length\ of\ the\ linked\ list\ is : 2\ and\ we\ want\ to\ insert\ data\ at\ node$
$position : 3\ i.e. at\ n + 1\ node.$

$when\ i = 1\ \ < (3 - 1) i.e. 2\ and\ current\ is\ not\ equal\ to\ `nullptr` :$

```
current = current->next;
```

$i.e. current\ pointer's\ value\ is\ assigned\ to\ the\ value\ contained\ in$

$current \rightarrow next, i.e. address\ of\ next\ node.$




$Now,$

```
i++;
```

$i.e. i = i + 1 = 1 + 1 = 2.$

$And\ , while(2 < 2\ \ [False] AND\ current! = nullptr\ [True])\ is\ flase\ , hence\ loop\ terminates\ here.$

$Now, if\ current's\ value\ would\ have\ been\ nullptr\ then:$

```
        if (current == nullptr)
        {
            cout << "Error: Invalid position." << endl;
            return;
        }
```

*This code is used to check if the pointer `current` is a null pointer (`nullptr`). If `current` is nullptr, it means that it does not point to any memory location(nor any arbritrary memory location)or any object rather to a special null value.*

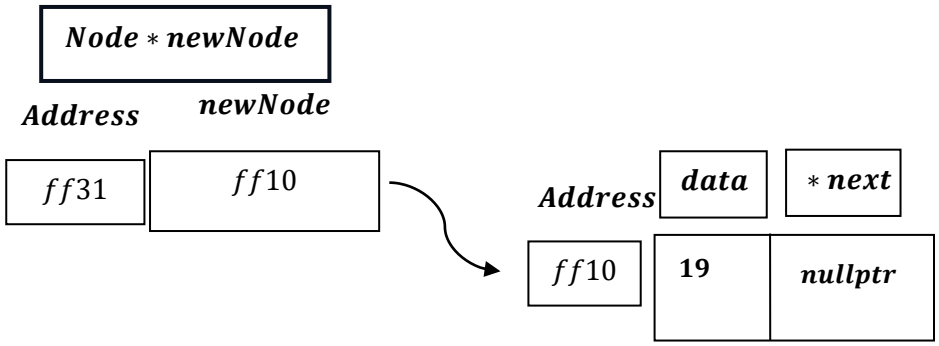
*In this case, the code displays an error message* `Error: Invalid position` *using the `cout` statement and then immediately `returns` from the InsertAtPosition method.*

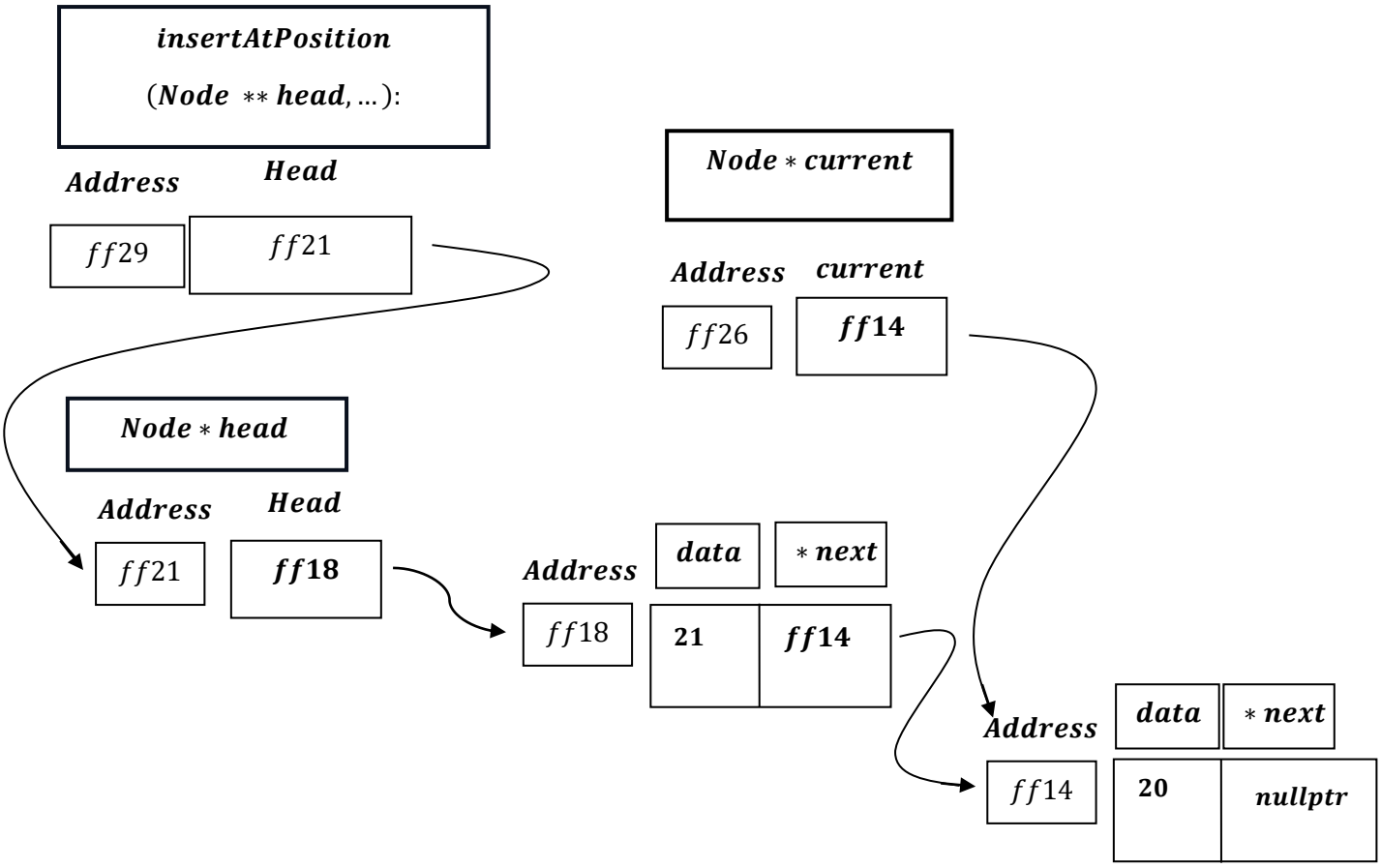
*Next*:

```
        else
        {
            newNode->next = current->next;
            current->next = newNode;
        }
```

*For*:

```
            newNode->next = current->next;
```



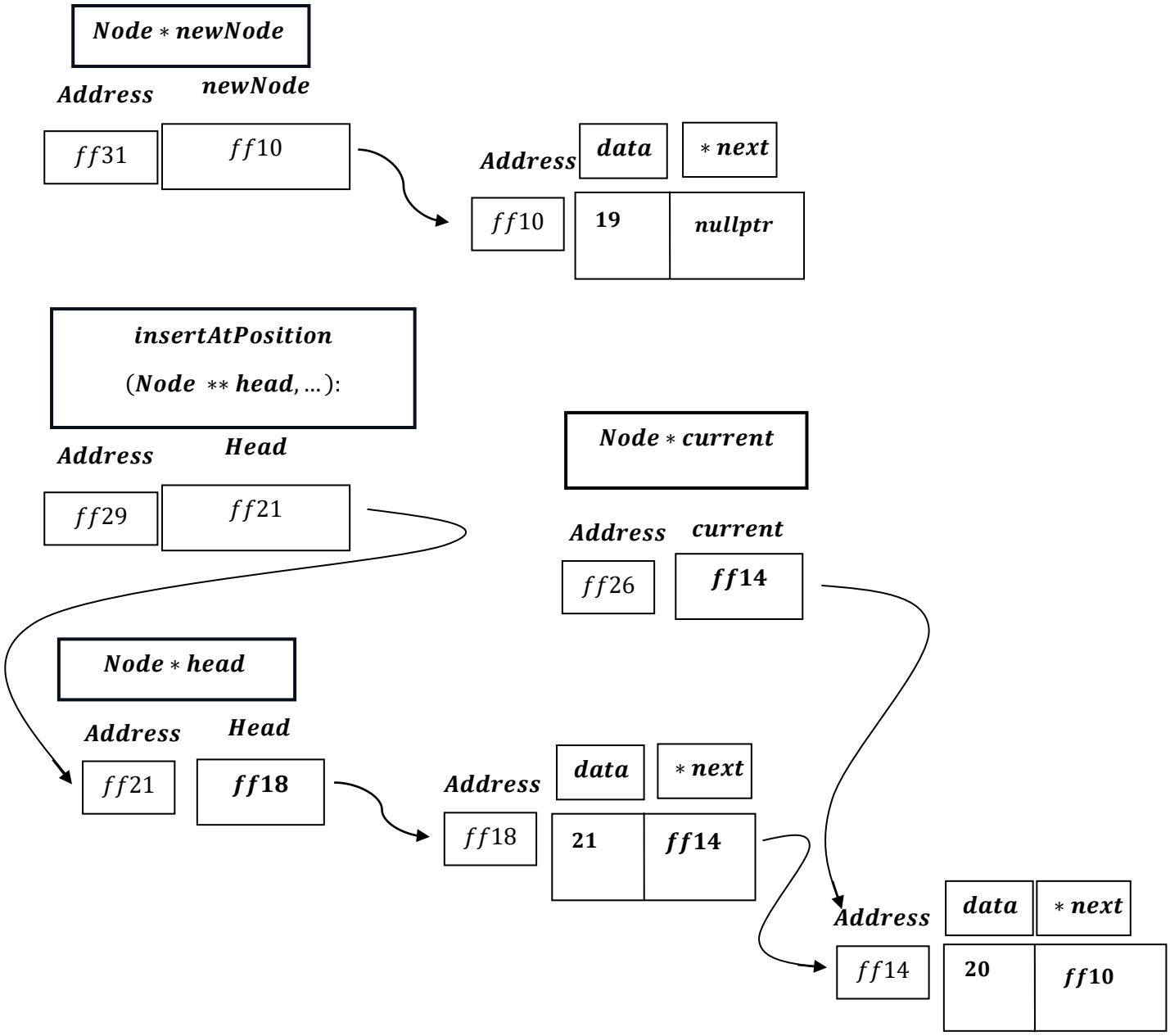



*i.e. newNode → next's value is assigned to the value contained in current → next. And as per current scenario current pointer of Node's structure pointed to Adrress containing next pointer value as `nullptr` .*

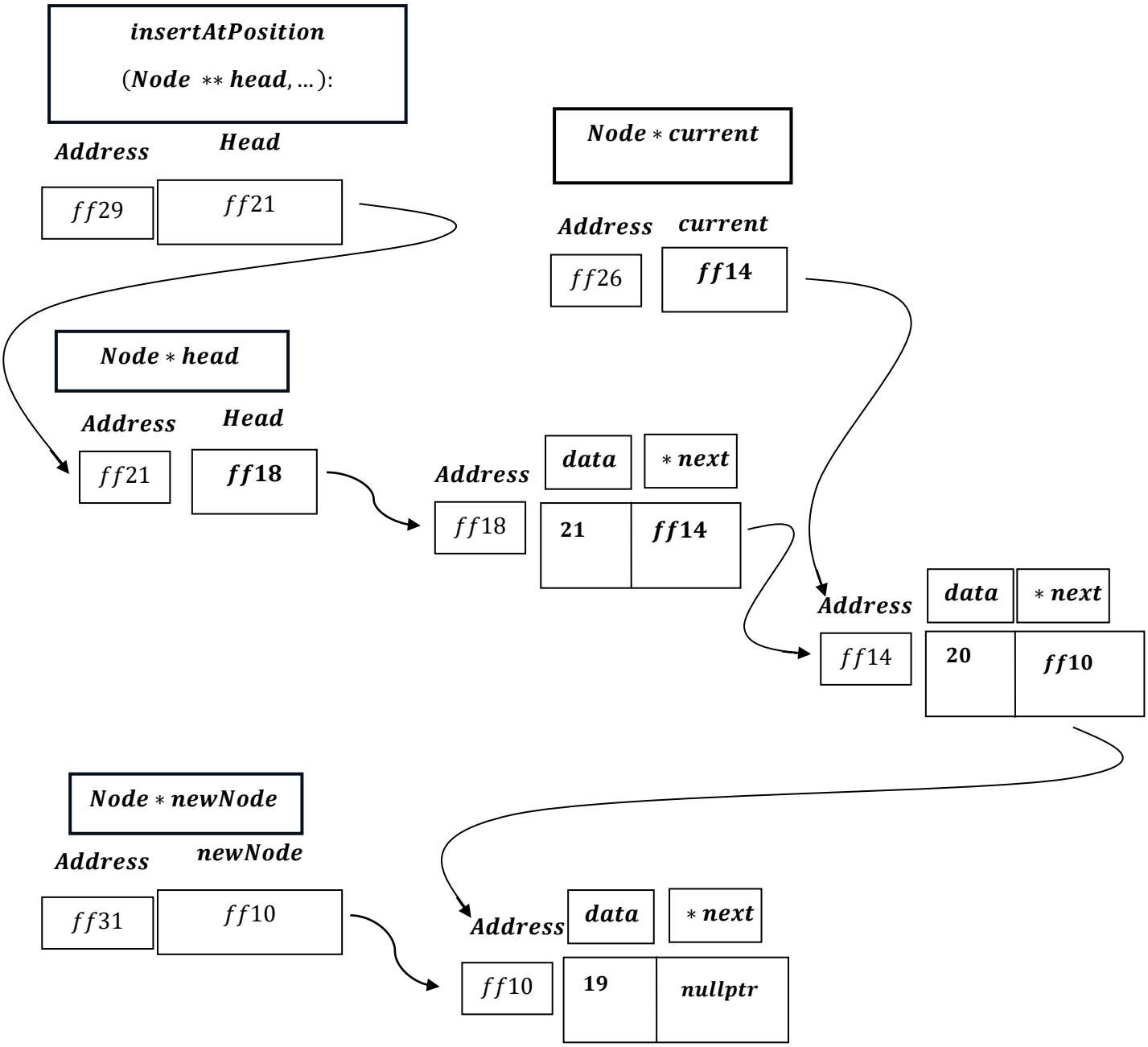
***Next***:

```
            current->next = newNode;
```

*i.e. current → next's value is assigned to the value contained in newNode pointer.*

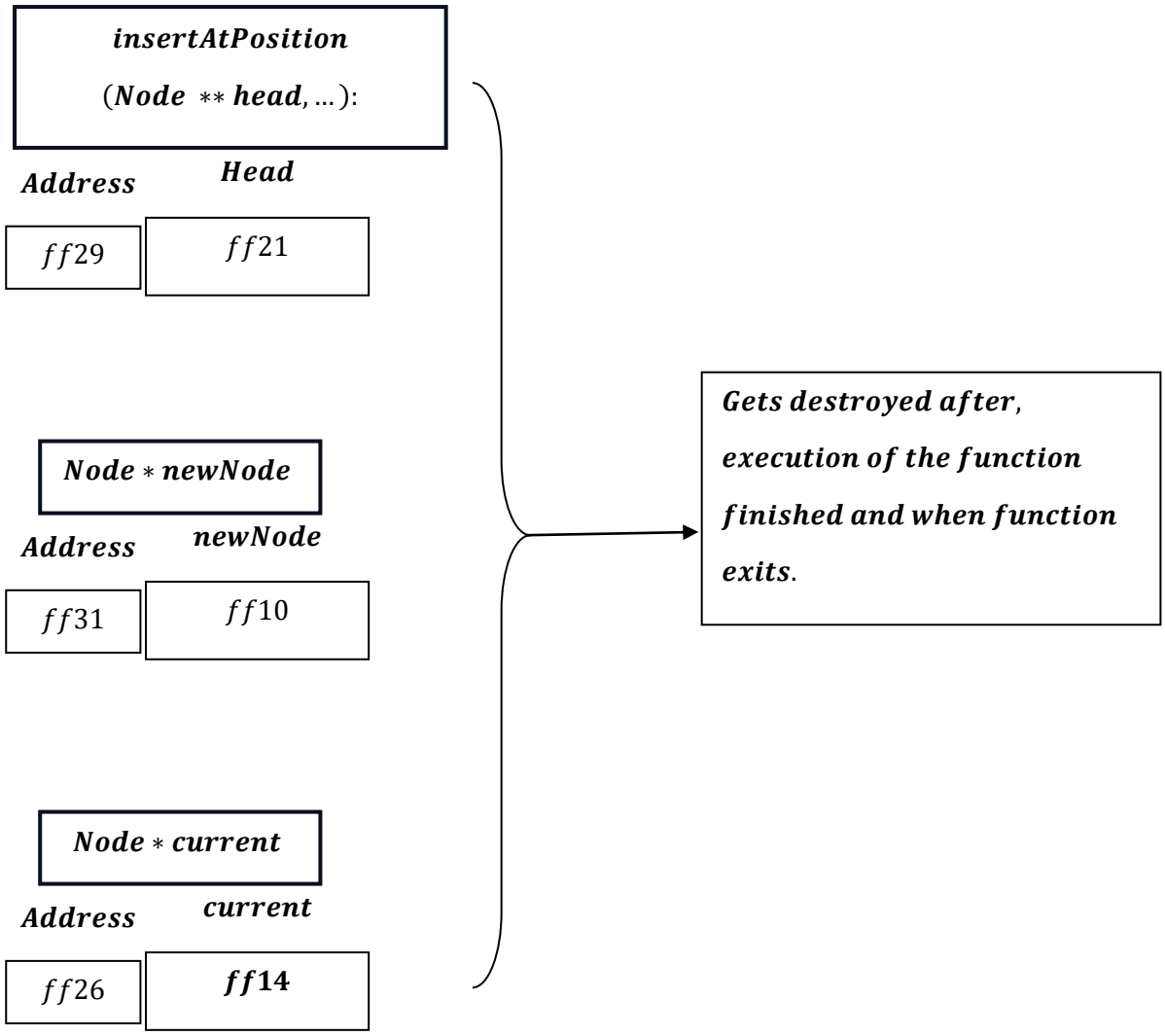



$current \rightarrow next = newNode = ff10[here]$ ***And then***:

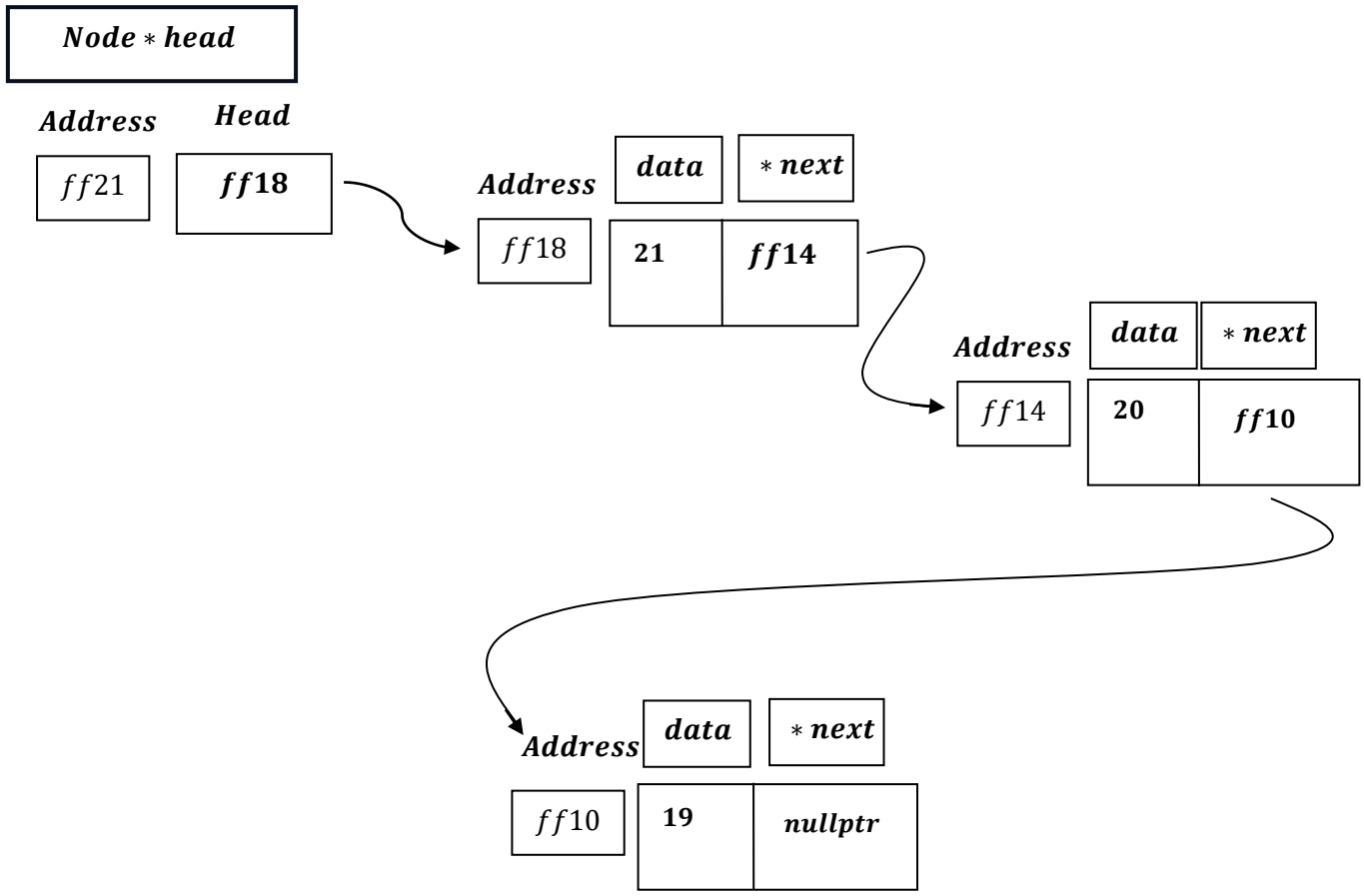



*Now, as Node * newNode , Node * current and Node ** head are temporary variable, therefore gets destroyed after function execution get finished .*

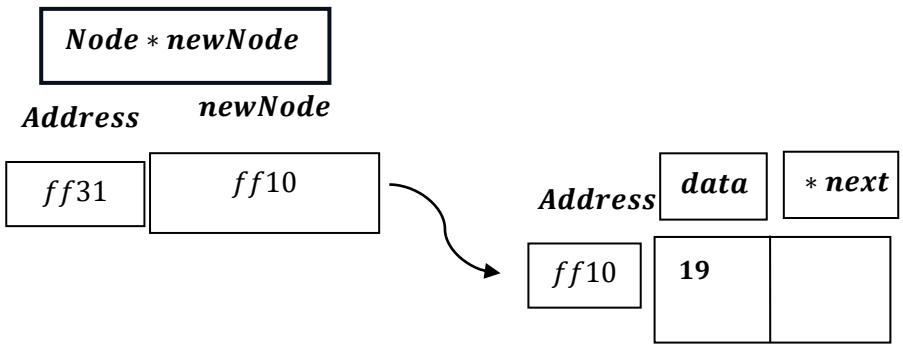
*i.e.*

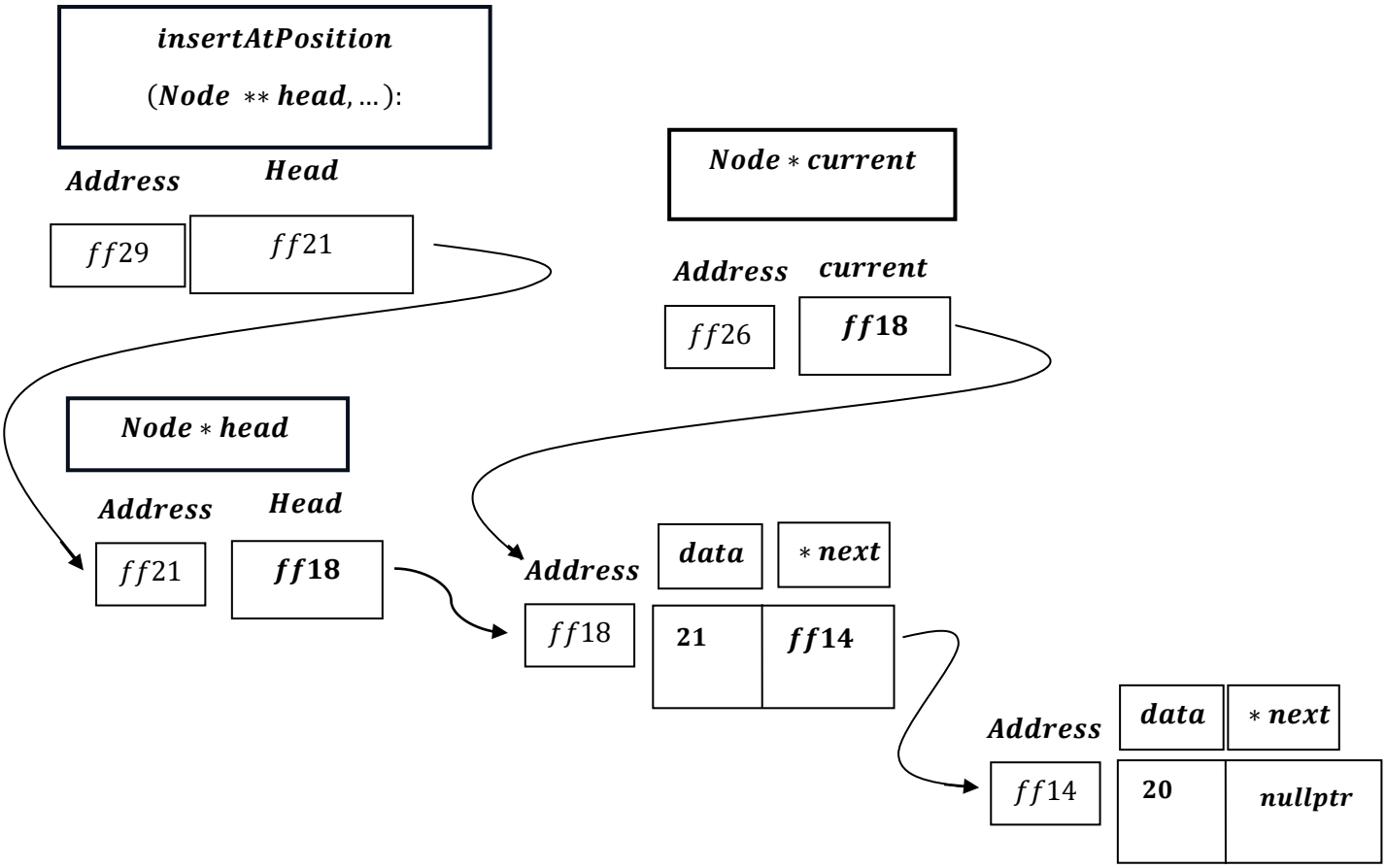



***Hence we are left with the linked list***:




***Now , does loop runs when position is 2? When position = 2, we have already this scenario before loop execution starts***:



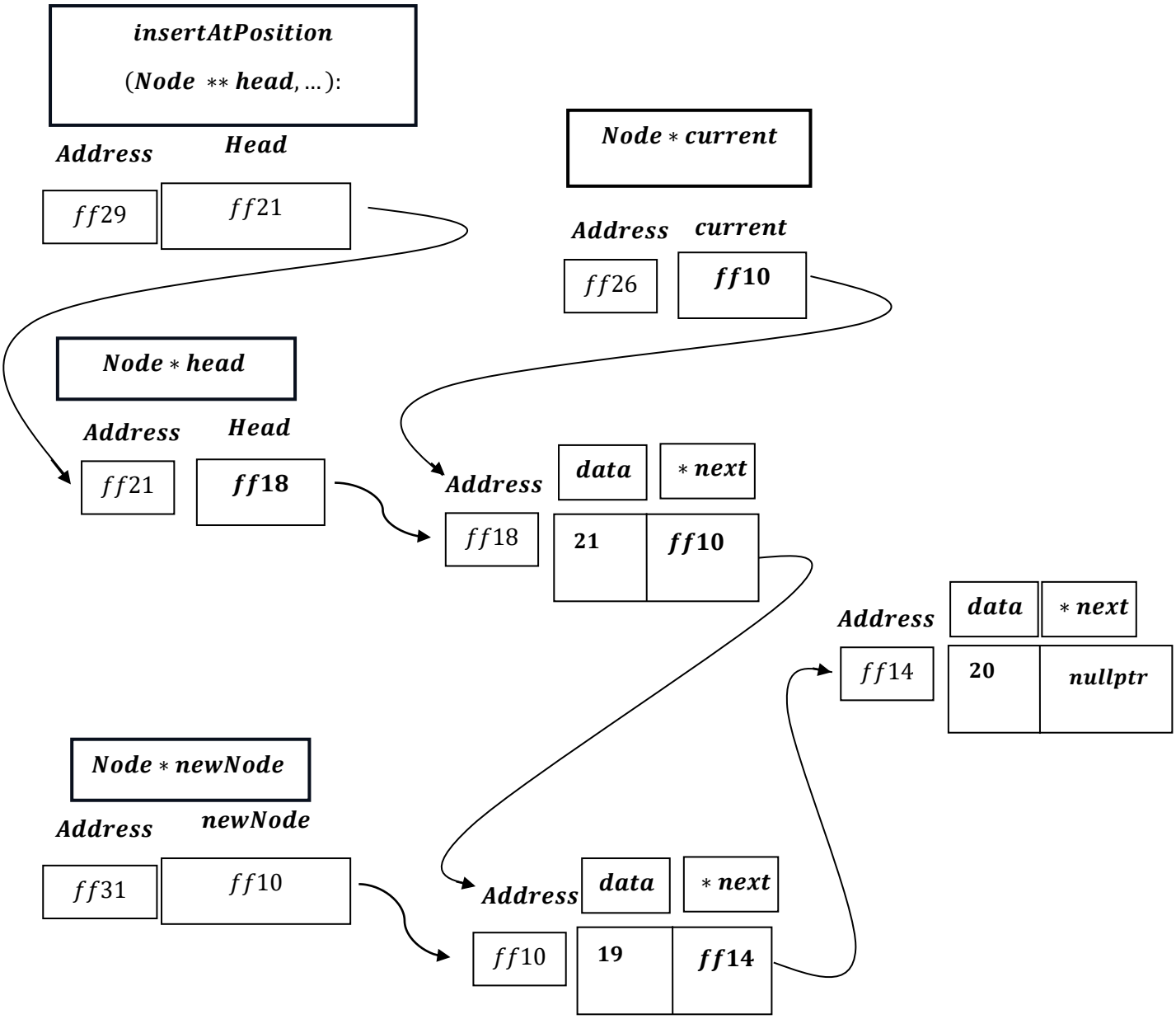



***As current is pointing at Node 1 before Node 2 , we donot need further loop to get executed , also:***

***While*** $(i = 1 < (2 - 1) = 1$ [***is False***] ***and current*** $! =$ ***nullptr is true***),
***the condition is false due to anding , hence directly we execute the else part:***

```
        else
        {
            newNode->next = current->next;
            current->next = newNode;
        }
```

***Which does is:***




***Hence , no loop execution happens , when Position is 1 or Position is 2 .***

---

***The `insertAtPosition` function inserts a new node with a given data value at a specified position in a singly linked list. Here's a step – by – step description of the process: –***

1. *Memory Allocation for New Node*:

– *The function starts by allocating memory for a new node using `malloc`.*

– *If memory allocation fails (`newNode == nullptr`), an error message is printed, and the function returns.*

2. *Assign Data to New Node*:

– *The data value is assigned to the `data` field of the new node.*

3. *Insert at the Beginning*:

– *If the specified position is `1`, the new node is inserted at the beginning of the list.*

– *The `next` pointer of the new node is set to the current head of the list.*

– *The head of the list is updated to point to the new node.*

– *The function then returns.*

4. *Check for Empty List*:

– *If the list is empty (`*head == nullptr`), an error message is printed, and the function returns.*

5. *Check for Invalid Position*:

– *If the specified position is less than `1`, an error message is printed, and the function returns.*

6. *Traverse to the Desired Position*:

– *A pointer `current` is initialized to point to the head of the list.*

– *A counter `i` is initialized to `1`.*

– *The function traverses the list until it reaches the node just before the desired position (`position − 1`).*

– If the end of the list is reached before reaching the desired position (`current == nullptr`), an error message is printed, and the function returns.

7. Insert at the Desired Position:

– The `next` pointer of the new node is set to point to the node currently at the desired position (`current–> next`).

– The `next` pointer of the node just before the desired position (`current`) is updated to point to the new node.

In summary, the function handles various cases such as inserting at the beginning, inserting in an empty list, and invalid positions. It ensures that the new node is correctly inserted at the specified position in the list.

---

# *Time Complexity of Insert At Position Function Module*

```
void insertAtPosition(Node **head, int data, int position)
{
    Node *newNode = (Node *)malloc(sizeof(Node));   → O(1)

    if (newNode == nullptr)   → O(1)
    {
        cout << "Error: Memory allocation failed." << endl;   → O(1)
        return;   → O(1)
    }

    newNode->data = data;   → O(1)

    if (position == 1)   → O(1)
    {
        newNode->next = *head;   → O(1)
        *head = newNode;   → O(1)
        return;   → O(1)
    }

    if (*head == nullptr)   → O(1)
    {
        cout << "Error: List is empty." << endl;   → O(1)
        return;   → O(1)
    }

    if (position < 1)   → O(1)
    {
        cout << "Error: Invalid position." << endl;   → O(1)
        return;   → O(1)
    }
    else
    {
        Node *current = *head;   → O(1)

        int i = 1;   → O(1)
```

```
        while (i < position - 1 && current != nullptr)
        {
            current = current->next;
            i++;
        }
        if (current == nullptr)
        {
            cout << "Error: Invalid position." << endl;
            return;
        }


        else
        {
            newNode->next = current->next;
            current->next = newNode;
        }
    }
}
```

$\rightarrow$ *runs `$n-1$` times at worst condition.*

$\rightarrow 1+1+\cdots$ *to* $n-1$ *times* $= O(n)$.

$\rightarrow 1+1+\cdots$ *to* $n-1$ *times* $= O(n)$.

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

*There are three case scenarios*:

$\rightarrow$ 1. *Position is* 1 *and* 2 *, i.e. we want to insert data at first or second .* (*Best Case*)

$\rightarrow$ 2. *Position is* $N+1$ *, i.e. we want to insert data position at end .* (*Worst Case*)

$\rightarrow$ 3. *Inserted position from* 3 *to* $N+1$, *while loop runs from* 1 *to* $N-1$.

*While if position is* 1 *and* 2 *, it takes constant amount of time* : $O(1)$.

*And when the position is uniformly distributed from* 1 *to* $(N+1)$.

(*Average Case i.e. Undetermined Position* ). *That is we donot whether the position will be* 1 *or* 2 *or* $N+1$, *where Node is to be inserted.*

## $\underline{1.\ When\ Position\ is\ 1\ and\ 2(Best\ Case)}$

$\underline{When\ Position\ is\ 1:}$

$\rightarrow$ $The\ function\ begins\ by\ allocating\ memory\ for\ a\ new\ node\ using\ malloc.$
$This\ operation\ takes\ constant\ time, O(1).$

$\rightarrow$ $After\ allocating\ memory, the\ function\ checks\ if\ the\ allocation\ was$
$successful. If\ not, it\ prints\ an\ error\ message\ and\ returns.$
$This\ step\ also\ takes\ constant\ time, O(1).$

$\rightarrow$ $If\ memory\ allocation\ is\ successful, the\ function\ initializes\ the\ new\ node's$
$data. This\ operation\ takes\ constant\ time, O(1).$

$\rightarrow$ $The\ function\ checks\ if\ the\ position\ is\ 1. If\ it\ is, the\ new\ node's\ next\ pointer$
$is\ set\ to\ the\ current\ head, and\ the\ head\ is\ updated\ to\ the\ new\ node.$
$This\ step\ takes\ constant\ time, O(1).$

$\rightarrow$ $The\ function\ checks\ if\ the\ head\ pointer\ is\ pointed\ to$ `null pointer` $and\ if\ the$
$check\ is\ true\ , then\ outputs\ \ a\ message\ i.e.\ list\ is\ empty\ and\ this\ takes\ constant$
$time\ , O(1).$

$\rightarrow$ $if\ the\ \ position\ is\ less\ than\ 1, then\ it\ outputs\ message$ `Invalid Position`
$which\ takes\ constant\ time\ , O(1).$

$Total\ time\ complexity: -$

$$O(1) + O(1) + O(1) + O(1) + O(1) + O(1) = O(1).$$

$We\ can\ say\ when\ we\ position\ is\ 1\ , we\ get\ the\ best\ time\ complexity:\ \Omega(1).$

***When Position is 2:***

→ *The function begins by allocating memory for a new node using malloc. This operation takes constant time, $O(1)$.*

→ *After allocating memory, the function checks if the allocation was successful. If not, it prints an error message and returns. This step also takes constant time, $O(1)$.*

→ *If memory allocation is successful, the function initializes the new node's data. This operation takes constant time, $O(1)$.*

→ *The function checks if the position is 0 and now the position is 1, hence if statement's condition checking will go false, the condition checking takes constant amount of time i.e. $O(1)$.*

→ *The function checks if the head pointer is pointed to `null pointer` and if the check is true , then outputs a message i.e. list is empty and this takes constant time , $O(1)$.*

→ *if the position is less than 0, then it outputs message `Invalid Position` which takes constant time , $O(1)$.*

→ *Now , position is nor 1 or head is neither `nullptr` , hence it goes to else part:*

→ *Now temporary current pointer of Node structure gets assigned to the value contained in head pointer which takes a constant amount of time i.e. $O(1)$.*

$\rightarrow$ *Temporary variable `i` gets assigned to `0` which takes constant amount of time i.e.* $O(1)$.

$\rightarrow$ *Now it initiates loop but it fails condition checking i.e.* *while* $(i = 1 < (2 - 1) = 1)$ *is false, which takes constant amount of time i.e.* $O(1)$.

$\rightarrow$ *Next it moves on to check current pointers value is nullptr which is not true here , this condition checking takes constant amount of time, i.e.* $O(1)$.

$\rightarrow$ *As current is not nullptr , it moves to else part, there at first it assigns value stored in next pointer of memory location pointed by newNode pointer of Node structure to next pointer of memory location pointed by current pointer of Node structure which takes constant amount of time i.e.* $O(1)$.

$\rightarrow$ *Secondly it assigns value stored in next pointer of memory location pointed by current pointer of Node structure to newNode pointer of Node structure which takes constant amount of time i.e.* $O(1)$.

*Total time complexity when Position is* $1: -$

$O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1)$
$+O(1) = O(1)$.

*Or, We can say when we position is* $2$ *, we get the best time complexity* : $\Omega(1)$.

---

## $\underline{2.\ When\ Position\ is\ N+1(Worst\ Case)}$

$\rightarrow$ $The\ function\ begins\ by\ allocating\ memory\ for\ a\ new\ node\ using\ malloc.$
$This\ operation\ takes\ constant\ time, O(1).$

$\rightarrow$ $After\ allocating\ memory, the\ function\ checks\ if\ the\ allocation\ was$
$successful. If\ not, it\ prints\ an\ error\ message\ and\ returns.$
$This\ step\ also\ takes\ constant\ time, O(1).$

$\rightarrow$ $If\ memory\ allocation\ is\ successful, the\ function\ initializes\ the\ new\ node's$
$data. This\ operation\ takes\ constant\ time, O(1).$

$\rightarrow$ $The\ function\ checks\ if\ the\ head\ of\ the\ list\ is\ null, indicating\ an\ empty\ list.$
$If\ the\ list\ is\ empty, it\ prints\ an\ error\ message\ and\ returns.$
$This\ step\ takes\ constant\ time, O(1).$

$\rightarrow$ $The\ function\ checks\ if\ the\ position\ is\ less\ than\ 1.$
$If\ it\ is, it\ prints\ an\ error\ message\ and\ returns.$
$This\ step\ takes\ constant\ time, O(1).$

$\rightarrow$ $If\ the\ position\ is\ valid\ and\ the\ list\ is\ not\ empty, the\ function\ uses\ a$
$loop\ to\ traverse\ the\ list\ and\ find\ the\ node\ just\ before\ the\ desired$
$position. The\ number\ of\ iterations\ required\ is\ equal\ to\ position\ -\ 1.$
$In\ the\ worst\ case:$

$\rightarrow$ $If\ the\ position\ is\ 1, the\ loop's\ inner\ statement\ runs\ 0\ time.$
$\rightarrow$ $If\ the\ position\ is\ 2, the\ loop's\ inner\ statement\ runs\ 0\ time.$
$\rightarrow$ $If\ the\ position\ is\ 3, the\ loop's\ inner\ statement\ runs\ 1\ times.$

$\rightarrow$ $If\ the\ position\ is\ 4, the\ loop's\ inner\ statement\ runs\ 2\ times.$

.

.

.

$\rightarrow$ $If\ the\ position\ is\ (N+1), the\ loop's\ inner\ statement\ runs: N-1\ times$ ,
$As\ when\ i = N < (N+1) - 1 = N\ becomes\ false$ , $hence\ no\ execution\ of$
$loop's\ inner\ statements.$

$Hence\ at\ worst\ case\ for\ position\ N+1\ \ loop\ run\ is\ \ N-1, hence$
$for\ inner\ loop\ statements$:

$current\ =\ current \rightarrow next;$
$i++;$

$Also\ runs\ N-1\ times\ each$ , $hence, for\ two\ statements\ we\ can\ say, 2(N-1)$
$and\ applying\ Big\ O\ i.e. O\big(2(N-1)\big) = O(N-1) = O(N)\ amount\ of\ time$
$approx$ .

$\rightarrow$ $Once\ the\ loop\ reaches\ the\ desired\ position, the\ new\ node\ is\ inserted\ by$
$adjusting\ the\ next\ pointers. This\ operation\ takes\ constant\ time, O(1).$

$Overall\ Time\ Complexity$:
$O(1)\ +\ O(1)\ +\ O(1)\ +\ O(1)\ +\ O(1)\ +\ O(n)\ +\ O(1)\ =\ O(n)$

$Hence\ worst\ time\ complexity\ is: O(n).$

---

$$3.\ Inserted\ position\ from\ 1\ to\ N+1, while\ loop\ runs\ from\ 1\ to\ N.$$

$$And\ when\ the\ position\ is\ uniformly\ distributed\ from\ 1\ to\ (N+1):$$

$$(Average\ Case\ i.e.\ Undetermined\ Position)$$

$\rightarrow$ $If\ the\ position\ is\ 1, the\ loop's\ inner\ statement\ runs\ 0\ time.$

$\rightarrow$ $If\ the\ position\ is\ 2, the\ loop's\ inner\ statement\ runs\ 0\ time.$

$\rightarrow$ $If\ the\ position\ is\ 3, the\ loop's\ inner\ statement\ runs\ 1\ times.$

$\rightarrow$ $If\ the\ position\ is\ 4, the\ loop's\ inner\ statement\ runs\ 2\ times.$

.

.

$\rightarrow$ $If\ the\ position\ is\ (N+1), the\ loop's\ inner\ statement\ runs: N-1\ times$ ,

$As\ when\ i = N < N\ becomes\ false$ , $hence\ no\ execution\ of\ loop's\ inner\ statements.$

$$Hence\ , \sum_{i=1}^{N-1} i = \frac{N(N-1)}{2}$$

$Now, as\ we\ take\ all\ the\ possibilites\ then$ , $we\ take\ the\ best\ case\ also\ i.e.$ $O(1), when\ position = 1\ and\ 2. Considering\ `T(P)`\ is\ time\ complexity\ of\ position\ `P`$ .

$$T(P) = \begin{cases} 1 & , when\ P = 1. \\ 1 & , when\ P = 2. \\ P-1 & , when\ P = 2, 3, 4, .., N. \end{cases}$$

$Now, If\ there\ is\ `N`\ no. of\ nodes\ there\ we\ can\ insert\ data\ at\ N+1\ positions.$

$$\boldsymbol{For\ position\ 1, the\ probability\ of\ insertion\ is \frac{1}{N+1}}.$$

$$\boldsymbol{For\ position\ 2, the\ probability\ of\ insertion\ \ is \frac{1}{N+1}}.$$

.

.

$$\boldsymbol{For\ position\ N+1, the\ probability\ of\ insertion\ \ is \frac{1}{N+1}}.$$

$$\boldsymbol{Hence\ probability\ of\ insertion\ will\ always\ remain\ \frac{1}{N+1}}.$$

$$\boldsymbol{\therefore\ Average\ Time\ Complexity\ \ is:\ probability\ of\ insertion\ \times (number\ of\ possible\ iterations + constant\ time\ complexities\ for\ left\ over\ positions)}\ .$$

$$= \left(\frac{1}{N+1} \times 1 + \frac{1}{N+1} \times 2 + \cdots + \frac{1}{N+1} \times N - 1\right) + \frac{1}{N+1} \times 1 + \frac{1}{N+1} \times 1$$

$$= \frac{1}{N+1} \times \{(1 + 2 + 3 + \cdots + N - 1) + 1 + 1\}$$

$$= \frac{1}{N+1} \times \left\{\sum_{i=1}^{N-1} i + 2\right\}$$

$$= \frac{1}{N+1} \times \left(\frac{N(N-1)}{2} + 2\right)$$

$$= \frac{N(N-1)}{2(N+1)} + \frac{2}{N+1}$$

$$= \frac{N^2 - N}{2N+2} + \frac{2}{N+1}$$

$\boldsymbol{Appplying\ Big\ Theta\ in\ the\ equation}$:

$$= \Theta\left(\frac{N^2 - N}{2N + 2}\right) + \Theta\left(\frac{2}{N + 1}\right)$$

$$= \Theta\left(\frac{N^2 - N}{2N + 2}\right) + \Theta\left(\frac{2}{N + 1}\right)$$

$\boldsymbol{Now, taking\ out\ the\ dominant\ term\ of\ numerator\ and\ denominator}$:

$$= \Theta\left(\frac{N^2}{2N}\right) + \Theta\left(\frac{2}{N}\right)$$

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{2}{N}\right)$$

$$= \Theta\left(\frac{N}{2}\right) + 2 \times \Theta\left(\frac{1}{N}\right)$$

$\boldsymbol{Now\ removing\ the\ constant\ `2`\ we\ get}$:

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

$$= \frac{1}{2} \times \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

$\boldsymbol{Now\ removing\ the\ constant\ `\frac{1}{2}`\ we\ get}$:

$$= \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

$When\ combining\ two\ Big\ Theta\ \Theta$ **terms , largest growth rate is the result**:

$\rightarrow \Theta(N)\ grows\ linearly\ with\ `N`$ .

$\rightarrow \Theta\left(\frac{1}{N}\right)\ decreases\ to\ a\ constant\ , i.e.\ when\ N\ grows\ larger\ , \frac{1}{N}\ approaches\ to\ 0.$

$This\ we\ can\ prove\ by\ limit\ and\ continuity\ also$:

$$\lim_{n\to\infty}\left(\frac{1}{n}\right) \Rightarrow \frac{1}{\infty} = 0\ \left[\frac{c}{\infty} = 0\ , where\ c\ is\ constant\right].$$

$$As\ \Theta(N)'s\ growth\ rate\ is\ higher\ , therefore\ , average\ case\ is\ \Theta(N).$$

$$Hence, Average\ Time\ Complexity\ is:\ \Theta(N).$$

---

$Note\ , by\ relation: \Omega \leq \Theta \leq O\ , we\ get$:

$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$

$\Rightarrow c_1 \times 1 \leq \frac{N(N-1)}{2(N+1)} + \frac{2}{N+1} \leq c_2 \times N$

---

$$Average\ Case\ Time\ Complexity[Most\ Simplest\ Alternative\ Apporach]:$$

$$(Simplest\ approach)$$

$For\ position\ 1, the\ probability\ of\ insertion\ is \frac{1}{N+1}.$

$For\ position\ 2, the\ probability\ of\ insertion\ is \frac{1}{N+1}.$

.

.

$$\boldsymbol{For\ position\ N+1, the\ probability\ of\ insertion\ \ is \frac{1}{N+1}}.$$

$$\boldsymbol{Hence\ probability\ of\ insertion\ will\ always\ remain\ \frac{1}{N+1},}$$

$$\boldsymbol{for\ position\ 1\ to\ N+1}.$$

$$\boldsymbol{Average\ case\ complexity:\ Probability\ of\ insertion\ \times \begin{pmatrix} Summation \\ of\ number\ of \\ Position. \end{pmatrix}}$$

$$= \frac{1}{N+1} \times \left( \sum_{i=1}^{N+1} i \right)$$

$$= \frac{1}{N+1} \times \left( \frac{1}{2}(N+1)\big((N+1)+1\big) \right) \left[ As, \sum_{k=1}^{n} k = \frac{1}{2}(n)(n+1) \right]$$

$$= \frac{1}{N+1} \times \left( \frac{(N+1)(N+2)}{2} \right)$$

$$= \frac{N+2}{2}$$

$$= \frac{N}{2} + \frac{2}{2}$$

$$= \frac{N}{2} + 1$$

$$\boldsymbol{Applying\ Big\ Theta\ \Theta\ we\ get}:$$

$$= \Theta\left(\frac{N}{2}\right) + \Theta(1)$$

$$= \frac{1}{2} \times \Theta(N)$$

$$= \Theta(N)$$

$$\textit{Hence, Average Time Complexity is}: \ \Theta\left(\frac{N}{2} + 1\right) \ \textit{or} \ \Theta(N).$$

---

*Now,*

*Note , by relation*: $\Omega \leq \Theta \leq O$ , *we get*:

$$\Longrightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Longrightarrow c_1 \times 1 \leq \frac{N}{2} + 1 \leq c_2 \times N$$

---

| *Singly Linked List* | *Cases* | *Time Complexity* |
|---|---|---|
| *InsertAt Position* | 1. *Best Case* | $\Omega(1)$ |
| | 2. *Worst Case* | $O(n)$ |
| | 3. *Average Case* | $\Theta(n)$ |

**********